

SHOPFORGE

Step-by-Step Development Guide

Spring Boot 3.2 + PostgreSQL 16 + React 18

8 Sprints | 16 Weeks | Senior Full-Stack Roadmap

PHASE 0: Prerequisites & Tooling

Required Software

Tool	Version	Install Command
Java JDK	21 (LTS)	sdkman: sdk install java 21-graalce
Maven	3.9+	sdkman: sdk install maven
Node.js	20 LTS	nvm install 20
PostgreSQL	16	brew install postgresql@16 / apt
Redis	7	brew install redis / apt
Docker Desktop	latest	docker.com/get-started
IntelliJ IDEA	2024+	jetbrains.com (Community OK)
VS Code	latest	With Extension Pack for Java
Postman / Bruno	latest	API testing
DBeaver / TablePlus	latest	DB GUI client
Git	2.40+	git-scm.com

Project Bootstrap — Spring Initializr

Go to start.spring.io and configure:

Setting	Value
Project	Maven
Language	Java
Spring Boot	3.2.x
Group	com.shopforge
Artifact	api
Packaging	Jar
Java	21

Add these dependencies:

- Spring Web
- Spring Data JPA
- PostgreSQL Driver
- Spring Security
- Validation
- Spring Boot Actuator
- Spring Boot DevTools
- Lombok
- MapStruct (add manually to pom.xml)
- SpringDoc OpenAPI (add manually)

Project Structure

```
shopforge-api/
├── src/main/java/com/shopforge/
│   ├── ShopforgeApplication.java
│   ├── common/
│   │   ├── exception/          # GlobalExceptionHandler, AppException
│   │   ├── response/          # ApiResponse, PageResponse wrappers
│   │   └── audit/              # AuditEntity base class
│   ├── config/
│   │   ├── SecurityConfig.java
│   │   ├── JwtConfig.java
│   │   ├── RedisConfig.java
│   │   └── OpenApiConfig.java
│   ├── auth/
│   │   ├── controller/AuthController.java
│   │   ├── service/AuthService.java
│   │   ├── dto/                # LoginRequest, RegisterRequest, TokenResponse
│   │   └── security/           # JwtFilter, JwtUtil, CustomUserDetailsService
│   └── user/
│       ├── model/User.java, UserRole.java
│       ├── repository/UserRepository.java
│       ├── service/UserService.java
│       ├── controller/UserController.java
│       └── dto/
```

```

├── product/
│   ├── model/           # Product, ProductVariant, ProductImage, Category, Brand
│   ├── repository/
│   ├── service/
│   ├── controller/
│   └── dto/
├── inventory/
├── cart/
├── order/
├── payment/
├── review/
├── coupon/
├── notification/
├── admin/
├── src/main/resources/
│   ├── application.yml
│   ├── application-dev.yml
│   ├── application-prod.yml
│   └── db/migration/    # Flyway SQL scripts
├── src/test/
└── pom.xml

```

application.yml (Base Config)

```

spring:
  application:
    name: shopforge-api
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/shopforge}
    username: ${DB_USER:shopforge}
    password: ${DB_PASS:shopforge}
  jpa:
    hibernate:
      ddl-auto: validate    # Flyway owns the schema
      show-sql: false
    properties:
      hibernate.jdbc.time_zone: UTC
      hibernate.order_updates: true
      hibernate.batch_size: 30
  flyway:
    enabled: true
    locations: classpath:db/migration
  data:
    redis:
      host: ${REDIS_HOST:localhost}
      port: 6379

jwt:
  secret: ${JWT_SECRET}    # min 256-bit base64 string
  access-token-expiry: 900  # 15 minutes (seconds)
  refresh-token-expiry: 604800 # 7 days

app:
  cors:
    allowed-origins: ${CORS_ORIGINS:http://localhost:3000}
  aws:
    bucket: ${S3_BUCKET:shopforge-media}
    region: ${AWS_REGION:ap-south-1}
  razorpay:
    key-id: ${RAZORPAY_KEY}
    key-secret: ${RAZORPAY_SECRET}

springdoc:
  api-docs.path: /api-docs

```

```
swagger-ui.path: /swagger-ui.html
```

Docker Compose (Local Dev)

```
version: '3.9'
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: shopforge
      POSTGRES_USER: shopforge
      POSTGRES_PASSWORD: shopforge
    ports: ['5432:5432']
    volumes: ['pgdata:/var/lib/postgresql/data']
  redis:
    image: redis:7-alpine
    ports: ['6379:6379']
  mailhog:
    image: mailhog/mailhog
    ports: ['1025:1025', '8025:8025'] # SMTP + web UI
volumes:
  pgdata:
```

Sprint 1: Auth & User Foundation (Week 1-2)

Goal: JWT login/register, role system, refresh tokens working end-to-end

Step 1.1 — Flyway: V1__create_enums_and_users.sql

```
CREATE TYPE user_status AS ENUM ('ACTIVE', 'BANNED', 'DELETED');
CREATE TYPE user_role AS ENUM ('CUSTOMER', 'SELLER', 'ADMIN', 'SUPPORT');

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) NOT NULL,
  email_verified BOOLEAN NOT NULL DEFAULT false,
  password_hash VARCHAR(255),
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100),
  status user_status NOT NULL DEFAULT 'ACTIVE',
  failed_login_attempts SMALLINT NOT NULL DEFAULT 0,
  lockout_until TIMESTAMPTZ,
  last_login_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX users_email_idx ON users (lower(email));

CREATE TABLE user_roles (
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role user_role NOT NULL,
  granted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, role)
);

CREATE TABLE refresh_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
token_hash VARCHAR(255) NOT NULL,
expires_at TIMESTAMPTZ NOT NULL,
revoked BOOLEAN NOT NULL DEFAULT false,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Step 1.2 — JPA Entities

User.java — key annotations:

```

@Entity @Table(name = "users")
@EntityListeners(AuditingEntityListener.class)
public class User {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(unique = true, nullable = false)
    private String email;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, columnDefinition = "user_status")
    private UserStatus status = UserStatus.ACTIVE;

    @CreationTimestamp private Instant createdAt;
    @UpdateTimestamp private Instant updatedAt;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private Set<UserRole> roles = new HashSet<>();
}

```

Step 1.3 — JWT Utility

```

@Component
public class JwtUtil {
    private final Key signingKey;
    private final long accessExpiry; // from config

    public String generateAccessToken(UUID userId, List<String> roles) {
        return Jwts.builder()
            .subject(userId.toString())
            .claim("roles", roles)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + accessExpiry * 1000))
            .signWith(signingKey)
            .compact();
    }

    public Claims extractClaims(String token) {
        return Jwts.parser().verifyWith((SecretKey) signingKey)
            .build().parseSignedClaims(token).getPayload();
    }
}

```

Step 1.4 — SecurityConfig

```

@Configuration @EnableWebSecurity @EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http, JwtFilter jwtFilter) throws
Exception {

```

```

return http
    .csrf(AbstractHttpConfigurer::disable)
    .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/v1/auth/**").permitAll()
        .requestMatchers(GET, "/api/v1/products/**").permitAll()
        .requestMatchers(GET, "/api/v1/categories/**").permitAll()
        .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated())
    .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
    .build();
}

@Bean
public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(12); }
}

```

Step 1.5 — Auth Endpoints to Implement

Endpoint	Body	Response	Notes
POST /api/v1/auth/register	{email, password, firstName}	201 {userId}	Send verification email
POST /api/v1/auth/login	{email, password}	200 {accessToken, refreshToken, user}	Check logout first
POST /api/v1/auth/refresh	{refreshToken}	200 {accessToken, refreshToken}	Rotate refresh token
POST /api/v1/auth/logout	{refreshToken}	204	Revoke refresh token in DB
POST /api/v1/auth/verify-email	{token}	200	OTP or signed link
POST /api/v1/auth/forgot-password	{email}	200	Send reset link
POST /api/v1/auth/reset-password	{token, newPassword}	200	Validate token expiry

Sprint 2: Product Catalog (Week 3-4)

Goal: Full product CRUD with variants, images, categories, search

Step 2.1 — Category Tree

Use recursive CTE for tree queries — avoid N+1 with JPA:

```
-- Get full tree with one query
```

```
WITH RECURSIVE cat_tree AS (
  SELECT id, parent_id, name, slug, 0 as depth
  FROM categories WHERE parent_id IS NULL AND is_active = true
  UNION ALL
  SELECT c.id, c.parent_id, c.name, c.slug, ct.depth + 1
  FROM categories c JOIN cat_tree ct ON c.parent_id = ct.id
  WHERE c.is_active = true
)
SELECT * FROM cat_tree ORDER BY depth, sort_order;
```

Step 2.2 — Product Entity Key Mappings

```
@Entity @Table(name = "products")
public class Product {
  @Id @GeneratedValue(strategy = GenerationType.UUID)
  private UUID id;

  @ManyToOne(fetch = FetchType.LAZY)
  @JoinColumn(name = "category_id", nullable = false)
  private Category category;

  @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)
  @OrderBy("sortOrder ASC")
  private List<ProductVariant> variants = new ArrayList<>();

  @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)
  @OrderBy("sortOrder ASC")
  private List<ProductImage> images = new ArrayList<>();

  @Enumerated(EnumType.STRING)
  private ProductStatus status = ProductStatus.DRAFT;

  @Column(columnDefinition = "tsvector", insertable = false, updatable = false)
  private String searchVector; // managed by DB trigger
}
```

Step 2.3 — Full-Text Search Repository

```
@Query(value = "SELECT p.* FROM products p " +
  "WHERE p.search_vector @@ plainto_tsquery('english', :query) " +
  "AND p.status = 'ACTIVE' AND p.deleted_at IS NULL " +
  "ORDER BY ts_rank(p.search_vector, plainto_tsquery('english', :query)) DESC",
  countQuery = "SELECT count(*) FROM products p WHERE p.search_vector @@
  plainto_tsquery('english', :query) AND p.status = 'ACTIVE' AND p.deleted_at IS NULL",
  nativeQuery = true)
Page<Product> searchProducts(@Param("query") String query, Pageable pageable);
```

Step 2.4 — Image Upload Flow

1. Receive multipart file in controller
2. Validate: type (jpg/png/webp), max size 5MB
3. Compress & convert to WebP via thumbnailator + webp-imageio
4. Upload to S3: products/{productId}/{uuid}.webp
5. Save URL to product_images table
6. Return CDN URL in response

Sprint 3: Cart, Wishlist & Addresses (Week 5-6)

Goal: Persistent cart with guest merge, address management

Step 3.1 — Cart Merge Strategy

```
// On login: merge guest cart into user cart
public void mergeGuestCart(String sessionId, UUID userId) {
    Cart guestCart = cartRepo.findBySessionId(sessionId).orElse(null);
    if (guestCart == null || guestCart.getItems().isEmpty()) return;

    Cart userCart = cartRepo.findActiveByUserId(userId)
        .orElseGet(() -> cartRepo.save(Cart.newForUser(userId)));

    for (CartItem guestItem : guestCart.getItems()) {
        userCart.addOrUpdateItem(guestItem.getVariant(), guestItem.getQuantity());
    }
    cartRepo.save(userCart);
    cartRepo.delete(guestCart);
}
```

Step 3.2 — Cart Stock Validation

Always validate stock before add and again at checkout (race condition window):

```
public void validateCartStock(Cart cart) {
    List<String> outOfStock = new ArrayList<>();
    for (CartItem item : cart.getItems()) {
        int available = inventoryRepo.findAvailableByVariantId(item.getVariant().getId());
        if (available < item.getQuantity()) {
            outOfStock.add(item.getVariant().getSku() + " (available: " + available + ")");
        }
    }
    if (!outOfStock.isEmpty()) {
        throw new InsufficientStockException(outOfStock);
    }
}
```

Sprint 4: Checkout & Payments (Week 7-8)

Goal: Full checkout flow, Razorpay integration, idempotent order creation

Step 4.1 — Checkout Service (Transactional)

```
@Service @Transactional
public class CheckoutService {

    public OrderResponse checkout(CheckoutRequest req, UUID userId) {
        // 1. Load and lock cart
        Cart cart = cartRepo.findActiveByUserIdWithLock(userId)
            .orElseThrow(() -> new CartNotFoundException());

        // 2. Validate stock (pessimistic lock inventory rows)
        validateAndReserveStock(cart);

        // 3. Calculate totals
        OrderTotals totals = calculateTotals(cart, req.getCouponCode());
    }
}
```

```
// 4. Create order record
Order order = Order.fromCart(cart, req, totals, userId);
orderRepo.save(order);

// 5. Initiate payment with gateway
PaymentInitResponse payInit = paymentService.initiate(order);

// 6. Clear cart
cart.clear();
cartRepo.save(cart);

return OrderResponse.of(order, payInit);
}
}
```

Step 4.2 — Razorpay Integration

Step	Action	API Call
1	Create Razorpay Order	POST https://api.razorpay.com/v1/orders
2	Return order_id to frontend	Frontend opens Razorpay checkout modal
3	Frontend pays	Razorpay calls our webhook
4	Verify webhook signature	HMAC-SHA256(order_id payment_id, key_secret)
5	Update payment status	Mark order as PAYMENT_RECEIVED
6	Trigger notification	Email + SMS to customer

Step 4.3 — Webhook Handler

```
@PostMapping("/api/v1/payments/webhook")
@Transactional
public ResponseEntity<?> handleWebhook(
    @RequestBody String payload,
    @RequestHeader("X-Razorpay-Signature") String signature) {

    if (!razorpayService.verifySignature(payload, signature)) {
        return ResponseEntity.status(401).build();
    }

    RazorpayEvent event = objectMapper.readValue(payload, RazorpayEvent.class);

    return switch (event.getEvent()) {
        case "payment.captured" -> {
            paymentService.confirmPayment(event.getPayload());
            yield ResponseEntity.ok().build();
        }
        case "payment.failed" -> {
            paymentService.failPayment(event.getPayload());
            yield ResponseEntity.ok().build();
        }
        default -> ResponseEntity.ok().build();
    };
}
```

Sprint 5: Order Management & Notifications (Week 9-10)

Goal: Order status machine, email/SMS notifications, return flow

Step 5.1 — Order State Machine

Use Spring State Machine or simple service-layer guard:

```
private static final Map<OrderStatus, Set<OrderStatus>> ALLOWED_TRANSITIONS = Map.of(
    PENDING, Set.of(PAYMENT_RECEIVED, PAYMENT_FAILED, CANCELLED),
    PAYMENT_RECEIVED, Set.of(PROCESSING, CANCELLED),
    PROCESSING, Set.of(SHIPPED, CANCELLED),
    SHIPPED, Set.of(DELIVERED, RETURN_REQUESTED),
    DELIVERED, Set.of(RETURN_REQUESTED, COMPLETED)
);

public void transitionOrder(UUID orderId, OrderStatus newStatus, UUID changedBy) {
    Order order = orderRepo.findByIdWithLock(orderId).orElseThrow();
    if (!ALLOWED_TRANSITIONS.getOrDefault(order.getStatus(), Set.of()).contains(newStatus)) {
        throw new InvalidStatusTransitionException(order.getStatus(), newStatus);
    }
    OrderStatus prev = order.getStatus();
    order.setStatus(newStatus);
    orderRepo.save(order);
    logStatusChange(orderId, prev, newStatus, changedBy);
    notificationService.sendOrderStatusUpdate(order, newStatus);
}
```

Step 5.2 — Email Notifications (Spring Mail)

```
@Service
public class EmailService {
    @Async
    public void sendOrderShipped(Order order) {
        MimeMessage msg = mailSender.createMimeMessage();
        MimeMessageHelper helper = new MimeMessageHelper(msg, true, "UTF-8");
        helper.setTo(order.getCustomerEmail());
        helper.setSubject("Your order #" + order.getOrderNumber() + " has shipped!");
        String html = templateEngine.process("order-shipped",
            ctx -> ctx.setVariable("order", order));
        helper.setText(html, true);
        mailSender.send(msg);
    }
}
```

Sprint 6: Reviews, Coupons, Admin Dashboard (Week 11-12)

Goal: UGC reviews, coupon engine, admin stats API

Step 6.1 — Review Service

```
public ReviewResponse createReview(CreateReviewRequest req, UUID userId) {
    // Check verified purchase
}
```

```

    boolean verified = orderItemRepo.existsByUserIdAndVariantProductId(userId,
req.getProductid());

    // One review per product per user
    if (reviewRepo.existsByProductIdAndUserId(req.getProductid(), userId)) {
        throw new DuplicateReviewException();
    }

    Review review = Review.builder()
        .productId(req.getProductid()).userId(userId)
        .rating(req.getRating()).body(req.getBody())
        .isVerifiedPurchase(verified)
        .status(ReviewStatus.PENDING) // auto-approve if no moderation needed
        .build();
    review = reviewRepo.save(review);
    // Trigger async rating recalculation
    applicationEventPublisher.publishEvent(new ReviewCreatedEvent(review));
    return ReviewResponse.from(review);
}

```

Step 6.2 — Coupon Validation Engine

```

public CouponValidationResult validateCoupon(String code, UUID userId, CartSummary cart) {
    Coupon coupon = couponRepo.findByCodeAndIsActive(code.toUpperCase(), true)
        .orElseThrow(() -> new CouponNotFoundException(code));

    // Time check
    Instant now = Instant.now();
    if (now.isBefore(coupon.getStartsAt()) || (coupon.getExpiresAt() != null &&
now.isAfter(coupon.getExpiresAt())))
        throw new CouponExpiredException();

    // Usage limit
    if (coupon.getUsageLimit() != null && coupon.getUsageCount() >= coupon.getUsageLimit())
        throw new CouponUsageLimitException();

    // Per-user limit
    if (coupon.getPerUserLimit() != null) {
        int userUsage = couponUsageRepo.countByUserIdAndCouponId(userId, coupon.getId());
        if (userUsage >= coupon.getPerUserLimit()) throw new CouponUserLimitException();
    }

    // Min order value
    if (coupon.getMinOrderValue() != null &&
cart.getSubtotal().compareTo(coupon.getMinOrderValue()) < 0)
        throw new CouponMinOrderException(coupon.getMinOrderValue());

    BigDecimal discount = computeDiscount(coupon, cart);
    return new CouponValidationResult(coupon, discount);
}

```

Sprint 7: Performance & Caching (Week 13-14)

Goal: Redis caching, N+1 elimination, query optimization

Step 7.1 — Redis Caching Strategy

Cache Key	TTL	Cached Data	Invalidation Trigger
<code>cat:tree</code>	30 min	Full category tree	Category create/update/delete
<code>prod:{id}</code>	10 min	Product detail (public fields)	Product update
<code>prod:list:{hash}</code>	5 min	Paginated product list	Any product in page changes
<code>user:{id}:cart:count</code>	5 min	Cart item count for badge	Cart add/remove
<code>search:{hash}</code>	2 min	Search results for repeated queries	Time-based only

Step 7.2 — Common N+1 Fixes

- Use `@EntityGraph` or `JOIN FETCH` for product + variants + images in single query
- Batch fetch user roles with `@BatchSize(size=30)` on collection
- Use Spring Projections (interface-based) for list views — load only needed columns
- Use `@QueryHints` with `HINT_FETCH_SIZE` for large result sets
- Enable Hibernate second-level cache (Caffeine) for Category and Brand (rarely change)

Step 7.3 — DB Connection Pool (HikariCP)

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 30000
      idle-timeout: 600000
      max-lifetime: 1800000
      pool-name: ShopForgeHikari
```

Sprint 8: Security, Testing & Deployment (Week 15-16)

Goal: Security hardening, test coverage, Docker image, staging deploy

Step 8.1 — Security Checklist

- Rate limiting: Bucket4j on `/auth/**` (5 req/min), `/api/**` (100/min guest)
- Input validation: `@Valid` on all request bodies, `@Size/@NotBlank` on all fields
- SQL injection: no native queries with string concat — use `:param` always
- CORS: whitelist only known origins — no wildcard in prod
- Security headers via Spring Security: HSTS, X-Content-Type, X-Frame-Options
- Secrets: no hardcoded creds — use env vars / AWS Secrets Manager

- Webhook signature verification before processing any payment event
- Idempotency keys on payment initiation — prevent double-charge

Step 8.2 — Testing Strategy

Layer	Type	Tool	Target Coverage
Services	Unit	JUnit 5 + Mockito	80%+
Repositories	Integration	Testcontainers (PostgreSQL)	All custom queries
Controllers	Slice	@WebMvcTest + MockMvc	All endpoints
Auth	Integration	TestRestTemplate	All auth flows
Payments	Integration	WireMock (Razorpay)	Success + failure paths
E2E	Full stack	Testcontainers + REST Assured	Critical user journeys

Step 8.3 — Dockerfile

```
# Build stage
FROM maven:3.9-eclipse-temurin-21-alpine AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn package -DskipTests -q

# Runtime stage
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S shopforge && adduser -S shopforge -G shopforge
USER shopforge
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java",
  "-XX:+UseContainerSupport",
  "-XX:MaxRAMPercentage=75",
  "-jar", "app.jar"]
```

Step 8.4 — GitHub Actions CI/CD

```
name: CI/CD
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env: { POSTGRES_DB: testdb, POSTGRES_USER: test, POSTGRES_PASSWORD: test }
```

```

    ports: ['5432:5432']
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with: { java-version: '21', distribution: 'temurin' }
    - name: Test
      run: mvn verify
    env:
      DB_URL: jdbc:postgresql://localhost:5432/testdb
      DB_USER: test
      DB_PASS: test
      JWT_SECRET: test-secret-min-256-bits-aaaaaaaaaaaaaaaaaa

  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build & push Docker image
        run: |
          docker build -t $ECR_URI:$GITHUB_SHA .
          docker push $ECR_URI:$GITHUB_SHA
      - name: Deploy to ECS
        run: aws ecs update-service --cluster prod --service api --force-new-deployment

```

PHASE 9: React Frontend

Setup

```

npm create vite@latest shopforge-web -- --template react-ts
cd shopforge-web
npm install @tanstack/react-query axios react-router-dom @tanstack/react-table
npm install zustand react-hook-form @hookform/resolvers zod
npm install tailwindcss @headlessui/react @heroicons/react
npx tailwindcss init -p

```

Folder Structure

```

src/
├── api/                                # Axios instances + auto-refresh interceptor
│   ├── client.ts
│   ├── auth.ts
│   ├── products.ts
│   └── orders.ts
├── components/                         # Shared UI components
├── features/                           # Feature-first organization
│   ├── auth/
│   ├── catalog/
│   ├── cart/
│   ├── checkout/
│   ├── orders/
│   └── admin/
├── hooks/                             # useCart, useAuth, useProduct
├── store/                             # Zustand stores
├── types/                             # TypeScript interfaces
└── utils/

```

Axios Auto-Refresh Interceptor

```
apiClient.interceptors.response.use(
  (res) => res,
  async (error) => {
    if (error.response?.status === 401 && !error.config._retry) {
      error.config._retry = true;
      const { data } = await axios.post('/api/v1/auth/refresh',
        { refreshToken: localStorage.getItem('refresh_token') });
      localStorage.setItem('access_token', data.accessToken);
      error.config.headers.Authorization = 'Bearer ' + data.accessToken;
      return apiClient(error.config);
    }
    return Promise.reject(error);
  }
);
```

Key Frontend Pages

Page	Route	Key Components
Home	/	Hero, FeaturedProducts, CategoryGrid, FlashSale
Product List	/c/:categorySlug	ProductGrid, FilterSidebar, SortDropdown, Pagination
Product Detail	/p/:slug	ImageGallery, VariantSelector, AddToCart, Reviews
Search	/search?q=	SearchResults, Filters, Autocomplete
Cart	/cart	CartItemList, OrderSummary, CouponInput
Checkout	/checkout	AddressForm, ShippingMethod, PaymentWidget
Order Success	/orders/:id/success	ConfirmationCard, Timeline
My Orders	/account/orders	OrderList, OrderDetail, ReturnFlow
Admin	/admin/*	Dashboard, Products, Orders, Users, Reports

Common Pitfalls & Senior Tips

Problem	Wrong Approach	Correct Approach
Money arithmetic	double/float for prices	BigDecimal everywhere — never float for money
Time zones	Store LocalDateTime	Store Instant/TIMESTAMP TZ (UTC); convert to user TZ only in response

Problem	Wrong Approach	Correct Approach
N+1 in JPA	Lazy load in loop	@EntityGraph / JOIN FETCH / batch size / Projections
Payment security	Trust client-side amount	Always calculate order total server-side; never trust req.amount
Stock race condition	Check then set in separate txn	SELECT ... FOR UPDATE on inventory row; atomic update
Password reset tokens	Store plain token in DB	Store SHA-256 hash; send raw token in email link only
ENUM in JPA	@Enumerated(EnumType.ORDINAL)	Always EnumType.STRING — ORDINAL breaks on enum reorder
Soft delete with JPA	Manual WHERE deleted_at IS NULL everywhere	@SQLRestriction("deleted_at IS NULL") on entity
Webhook replay	No idempotency check	Idempotency key in DB UNIQUE constraint; skip if already processed
Refresh token rotation	Reuse same refresh token	Invalidate old token on each use; detect reuse = revoke all user tokens